DEPARTMENT OF
COMPUTER SCIENCE AND ENGINEERING
UNIVERSITY OF DHAKA

Software Design Document 01 — Version 1.0

CSE-4113: Internet Programming Lab

BATCH: 4th YEAR 1ST SEMESTER 2025

Team Name: Formula1

COURSE INSTRUCTORS

Dr. Md. Mamun Or Rashid
Professor, Dept. of CSE, University of Dhaka

Mr. Md. Fahim Arefin
Lecturer, Dept. of CSE, University of Dhaka

Mr. Md. Ahasanul Alam
Lecturer, Dept. of CSE, University of Dhaka

Submitted By:

Amio Rashid
Roll 59

Syed Naimul Islam
Roll 37

Mahmud Hasan Walid
Roll 51

Md Al Habib
Roll 17

Contents

1 Introduction **5**

1.1 Purpose 5

1.2 Scope 5

1.3 Definitions and Acronyms 5

1.4 References 6

1.5 Project Repository 6

2 System Architecture **6**

2.1 Architectural Style 6

2.2 System Component Overview 7

2.3 API Gateway Design 7

2.4 Inter-Service Communication 8

2.5 Technology Stack 8

3 Microservice Designs **9**

3.1 MS1 — User & Auth Service 9

3.1.1 Responsibilities 9

3.1.2 Module Structure 9

3.1.3 API Endpoints 10

3.1.4 Database Schema — `auth` 11

3.1.5 JWT Design 12

3.1.6 Key Algorithm — Password Reset 12

3.1.7 Auth Flow Sequence Diagram 12

3.2 MS2 — Election Service (Critical Service) 13

3.2.1 Responsibilities 13

3.2.2 Module Structure 13

3.2.3 API Endpoints 14

3.2.4 Database Schema — `election` 14

3.2.5 Critical Algorithm — Vote Atomicity 16

3.2.6 Algorithm — Election Auto-Scheduling 18

3.3 MS3 — Event, Notice & Media Service 20

3.3.1 Responsibilities 20

3.3.2 Module Structure 20

3.3.3 API Endpoints 21
3.3.4 Database Schema — `content` 22
3.3.5 File Storage Design 23
3.4 MS4 — Finance, Notification & Logging Service 24
3.4.1 Responsibilities 24
3.4.2 Module Structure 24
3.4.3 API Endpoints 24
3.4.4 Database Schema — `finance` 25
3.4.5 BullMQ Worker Design 26

4 Database Design **27**
4.1 Schema Overview 27
4.2 Cross-Schema References 27
4.3 Consolidated Entity-Relationship Diagram 27

5 Asynchronous Communication **28**
5.1 BullMQ Architecture 28
5.2 Event Catalogue 28
5.3 BullMQ Event Flow Diagram 28

6 Security Design **29**
6.1 JWT Authentication Flow 29
6.2 RBAC Implementation 29
6.3 Vote Anonymity Design 30
6.4 Additional Security Measures 30

7 Deployment Architecture **31**
7.1 Docker Compose Layout 31
7.2 Service Port Summary 32
7.3 Database Initialisation 32
7.4 Failure Isolation 33

8 Document Revision History **33**

1 Introduction

1.1 Purpose

This Software Design Document (SDD) translates the requirements established in the CSEDU Students' Club Management System SRS v1.1 into concrete architectural and implementation decisions. It defines the system's microservice decomposition, database schemas, REST API contracts, module structures, asynchronous communication design, security model, and deployment layout. The intended audience is the four-member development team, each responsible for one microservice, along with academic reviewers.

1.2 Scope

This document covers the complete technical design for the CSEDU Students' Club Management System, decomposed into four microservices and one API Gateway:

- **MS1 — User & Auth Service** (Amio Rashid): registration, login, JWT issuance, RBAC, password reset, session management.
- **MS2 — Election Service** (Syed Naimul Islam): election lifecycle, concurrency-safe voting, candidate profiles, result concealment, auto-scheduling.
- **MS3 — Event, Notice & Media Service** (Md. Al Habib): event and volunteer management, notice board, media file handling.
- **MS4 — Finance, Notification & Logging Service** (Mahmud Hasan Walid): budget proposals, expenditure tracking, email delivery via BullMQ, activity audit logs.
- **API Gateway** (Amio Rashid): single entry point, JWT verification, rate limiting, request routing.

1.3 Definitions and Acronyms

Term	Definition
SDD	Software Design Document
MS1–MS4	Microservice 1 through 4 as defined in Section 2
API Gateway	Entry-point Express.js service that routes requests to microservices
BullMQ	Redis-backed job queue library for Node.js
RS256	RSA Signature with SHA-256 — asymmetric JWT signing algorithm
RBAC	Role-Based Access Control
Schema	A named PostgreSQL namespace used to logically isolate each service's tables

SFU	`SELECT FOR UPDATE` — a PostgreSQL row-level lock used in voting transactions
Docker Compose	Tool for defining and running multi-container Docker applications
Multer	Node.js middleware for handling `multipart/form-data` file uploads
Nodemailer	Node.js library for sending emails via SMTP

1.4 References

- Formula1 — Software Requirements Specification 01, Version 1.1, April 2026.
- BullMQ Documentation: https://docs.bullmq.io
- PostgreSQL 16 Documentation: https://www.postgresql.org/docs/16/

1.5 Project Repository

The complete source code and documentation for this project is available at:

https://github.com/SudinsHub/CSEDUSC-by-Formula1.git

All microservices, the API Gateway, database initialization scripts, and Docker Compose configuration are maintained in this repository.

2 System Architecture

2.1 Architectural Style

The system adopts a **microservices architecture** with domain-driven service boundaries. Each service owns its data, exposes a REST API over HTTP, and communicates asynchronously with other services through a BullMQ message queue backed by Redis. A single API Gateway acts as the sole entry point for the React frontend.

Key architectural properties:

- **Failure isolation**: a crash in MS3 (Event/Notice/Media) does not affect MS2 (Election), so voting continues uninterrupted during an active election.
- **Independent scalability**: MS2 can be horizontally scaled to additional containers during election periods without redeploying other services.
- **Stateless services**: all services are stateless; session state is carried in the JWT access token and refresh tokens stored in the database, enabling horizontal scaling behind a load balancer without session stickiness.

- **Schema isolation**: each service reads and writes only its own PostgreSQL schema (`auth`, `election`, `content`, `finance`), all hosted on one PostgreSQL instance to reduce operational complexity while maintaining logical ownership boundaries.

2.2 System Component Overview

Figure 1 shows the high-level component layout. The React frontend communicates exclusively with the API Gateway over HTTPS. The Gateway validates JWTs and routes requests to the appropriate downstream service via Docker internal HTTP. MS4 is the sole consumer of the BullMQ queues; all other services act as producers.

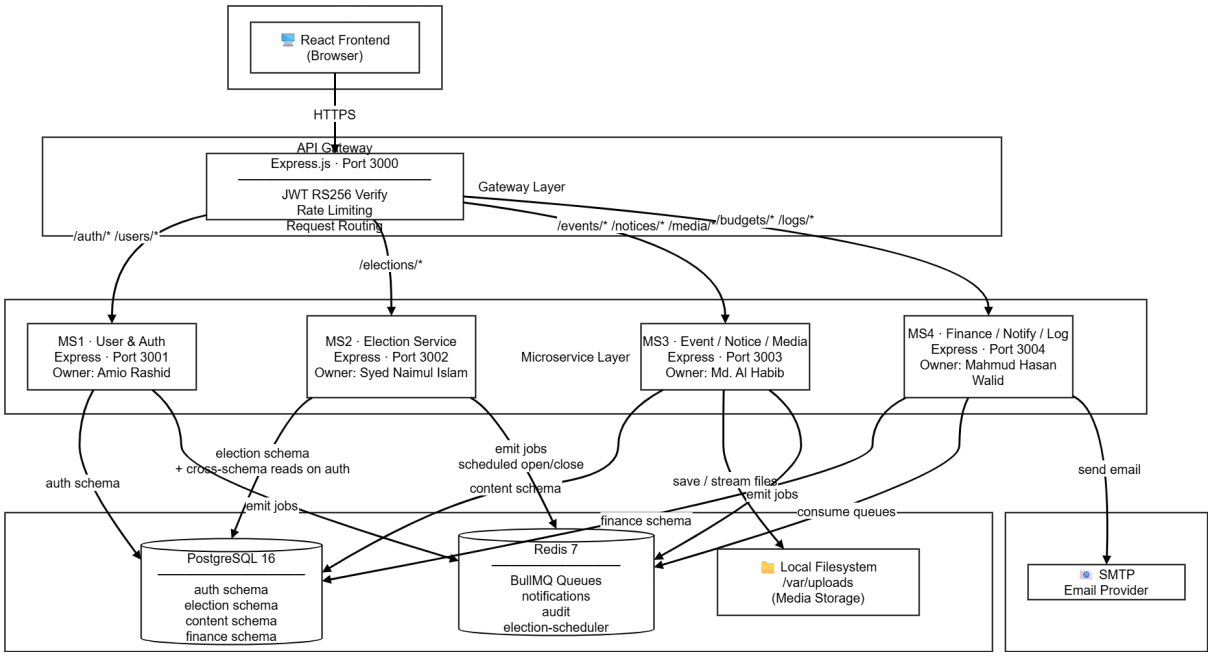

Figure 1: System Architecture Diagram

2.3 API Gateway Design

The API Gateway is a custom **Express.js** service assigned to Amio Rashid. It is the only component exposed to the public internet (port 443 in production, port 3000 in development).

Responsibilities:

- **TLS termination**: handled by an Nginx reverse proxy in production (outside the scope of this document); the Gateway itself listens on HTTP within the Docker network.
- **JWT verification**: on every request to a protected route, the Gateway calls `jwt.verify(token, RS256_PUBLIC_KEY)`. The RS256 public key is loaded from an environment variable at startup. No call to MS1 is needed per request — verification is fully local using the cached public key.

- **Header injection**: after successful verification, the Gateway attaches `X-User-Id`, `X-User-Role`, and `X-User-Email` to the forwarded request. Downstream services trust these headers unconditionally.
- **Rate limiting**: `express-rate-limit` is applied globally (100 requests / minute per IP) and at a stricter level on `POST /elections/:id/vote` (30 requests / minute per IP).
- **Request routing**: the Gateway uses `http-proxy-middleware` to forward requests to the correct internal service.

Route mapping:

Path Prefix	Target Service	JWT Required
`/auth/*`, `/auth/login`, `/auth/register`, `/auth/forgot-password`, `/auth/reset-password/*`	MS1 :3001	No
`/auth/logout`, `/auth/refresh`	MS1 :3001	Yes
`/users/*`	MS1 :3001	Yes (Admin)
`/elections/*`	MS2 :3002	Varies per route
`/events/*`, `/notices/*`, `/media/*`	MS3 :3003	Varies per route
`/budgets/*`, `/logs/*`	MS4 :3004	Yes

2.4 Inter-Service Communication

- **Synchronous (REST)**: the Gateway routes client requests to services via internal HTTP. Services do not call each other's REST APIs. Where one service needs data owned by another service (e.g., MS2 needs to verify a user's batch for election eligibility), it performs a **direct cross-schema SQL query** within the shared PostgreSQL instance (e.g., `SELECT batch_year FROM auth.users WHERE user_id = $1`). This avoids network hops while preserving logical ownership.
- **Asynchronous (BullMQ)**: MS1, MS2, MS3, and MS4's budget REST endpoint enqueue jobs to Redis-backed BullMQ queues. MS4 runs dedicated worker processes (`NotificationWorker`, `AuditWorker`) that consume these queues. This decouples email delivery and audit logging from the request critical path.

2.5 Technology Stack

Layer	Technology	Version	Justification
Frontend	React	18.x	SPA capability; team familiarity

API Gateway	Express.js	4.x	Lightweight; `http-proxy-middleware` integration
All Microservices	Express.js	4.x	Consistent Node.js stack; no framework overhead
Database client	node-postgres (`pg`)	8.x	Direct SQL; full control over transactions and locking
Database	PostgreSQL	16.x	ACID compliance; schema namespacing; `SELECT FOR UPDATE` support
Message queue	BullMQ	4.x	Redis-backed; delayed jobs; job persistence across restarts
Cache / broker	Redis	7.x	BullMQ dependency; optionally used for refresh-token blacklist
Authentication	jsonwebtoken (RS256)	9.x	Asymmetric signing; gateway can verify without calling MS1
Password hashing	bcrypt	5.x	Cost factor 12 per NFR6
File uploads	multer	1.x	Multipart parsing; streams directly to local filesystem
Email	Nodemailer + SMTP	6.x	Reliable; provider-agnostic
Containerisation	Docker + Compose	24.x	Service isolation; reproducible deploys; shared network

3 Microservice Designs

3.1 MS1 — User & Auth Service

Owner: Amio Rashid **Port:** 3001 **Database schema:** `auth`

3.1.1 Responsibilities

MS1 handles all identity concerns: new student registration, administrator approval workflow, JWT issuance (access + refresh tokens), RBAC role management, password reset, and session logout. It is also responsible for generating the RS256 keypair whose public key is distributed to the API Gateway at deployment time.

3.1.2 Module Structure

MS1 follows a four-layer architecture: **Router** → **Controller** → **Service** → **Repository**. Figure 2 shows the module diagram.

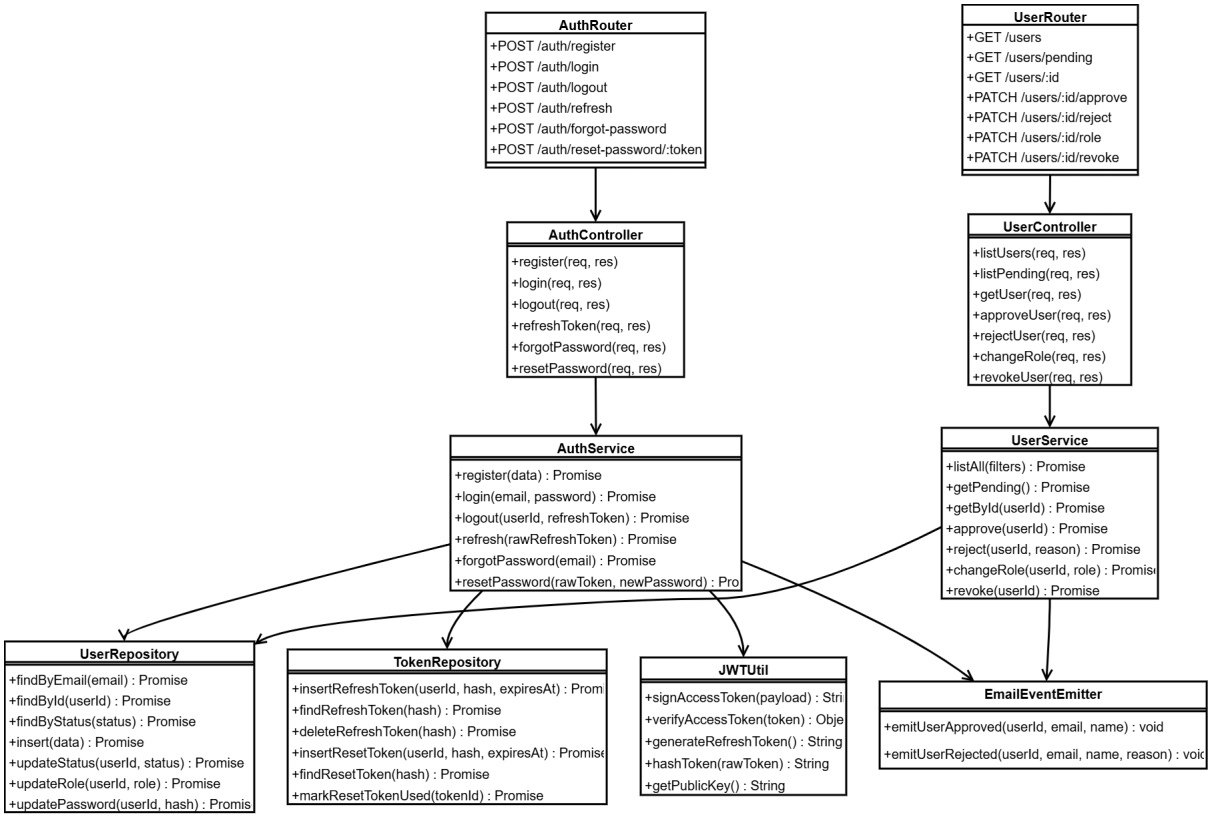

Figure 2: MS1 Module Diagram

3.1.3 API Endpoints

Method	Path	Auth	Role	Description
POST	`/auth/register`	No	—	Register new student
POST	`/auth/login`	No	—	Login; returns access + refresh token
POST	`/auth/logout`	Yes	Any	Invalidate refresh token
POST	`/auth/refresh`	No	—	Exchange refresh token for new access token
POST	`/auth/forgot-password`	No	—	Send password reset email
POST	`/auth/reset-password/:token`	No	—	Submit new password
GET	`/users`	Yes	Admin	List all users (with filters)
GET	`/users/pending`	Yes	Admin	List pending registrations

GET	`/users/:id`	Yes	Any	Get a user's public profile
PATCH	`/users/:id/approve`	Yes	Admin	Approve registration
PATCH	`/users/:id/reject`	Yes	Admin	Reject registration (with reason)
PATCH	`/users/:id/role`	Yes	Admin	Promote / change role
PATCH	`/users/:id/revoke`	Yes	Admin	Revoke membership

3.1.4 Database Schema — `auth`

Table: `auth.users`

Column	Type	Notes
`user_id`	`SERIAL PK`	Auto-incrementing primary key
`name`	`VARCHAR(120)`	Full name
`email`	`VARCHAR(200) UNIQUE`	Must end in `cs.du.ac.bd` or `cse.du.ac.bd`
`password_hash`	`TEXT`	bcrypt hash (cost 12)
`role`	`VARCHAR(20)`	`student`, `ec_member`, `admin`
`status`	`VARCHAR(20)`	`pending`, `active`, `rejected`, `revoked`
`batch_year`	`INTEGER`	Academic batch (e.g., 2021)
`created_at`	`TIMESTAMPTZ`	Registration timestamp
`updated_at`	`TIMESTAMPTZ`	Last update timestamp

Table: `auth.refresh_tokens`

Column	Type	Notes
`token_id`	`SERIAL PK`	—
`user_id`	`INTEGER FK`	References `auth.users`
`token_hash`	`TEXT UNIQUE`	SHA-256 hash of the raw refresh token
`expires_at`	`TIMESTAMPTZ`	Token lifetime: 7 days
`revoked`	`BOOLEAN`	Set to true on logout
`created_at`	`TIMESTAMPTZ`	—

Table: `auth.password_reset_tokens`

Column	Type	Notes
`token_id`	`SERIAL PK`	—
`user_id`	`INTEGER FK`	References `auth.users`
`token_hash`	`TEXT UNIQUE`	SHA-256 hash of the emailed token
`expires_at`	`TIMESTAMPTZ`	30 minutes from creation
`used`	`BOOLEAN`	Marked true after first successful use

created_at	TIMESTAMPTZ	—

3.1.5 JWT Design

- **Access token**: RS256, expires in **15 minutes**. Payload: { `sub: user_id, role, email, name, batch_year, iat, exp` }.
- **Refresh token**: a cryptographically random 64-byte hex string. The raw token is sent to the client; only its SHA-256 hash is stored in `auth.refresh_tokens`.
- **Key distribution**: the RS256 private key is stored in MS1's environment only. The public key is passed to the API Gateway as an environment variable (`JWT_PUBLIC_KEY`) at deploy time. No runtime network call is needed for the gateway to verify tokens.
- **Logout**: the refresh token hash is deleted from `auth.refresh_tokens`. The short-lived access token (max 15 min) expires naturally, which is acceptable given the window.

3.1.6 Key Algorithm — Password Reset

1. User calls `POST /auth/forgot-password` with their email.
2. `AuthService` generates a 32-byte cryptographically random token via `crypto.randomBytes(32).t`
3. The SHA-256 hash of this token is stored in `auth.password_reset_tokens` with `expires_at = NOW() + '30 minutes'` and `used = false`.
4. An email job is enqueued to BullMQ's `notifications` queue carrying the raw token embedded in the reset link.
5. User clicks the link: `POST /auth/reset-password/:rawToken`.
6. Server hashes the incoming token, queries `auth.password_reset_tokens` for a matching row where `used = false AND expires_at > NOW()`.
7. If found: bcrypt-hash the new password, update `auth.users`, mark the reset token as `used = true`. Old password is immediately invalid.
8. If not found or expired: return `400 Bad Request` with "Link expired or already used."

3.1.7 Auth Flow Sequence Diagram

Figure 3 shows the complete login and subsequent authenticated request flow, illustrating how the Gateway verifies tokens locally using the cached public key.

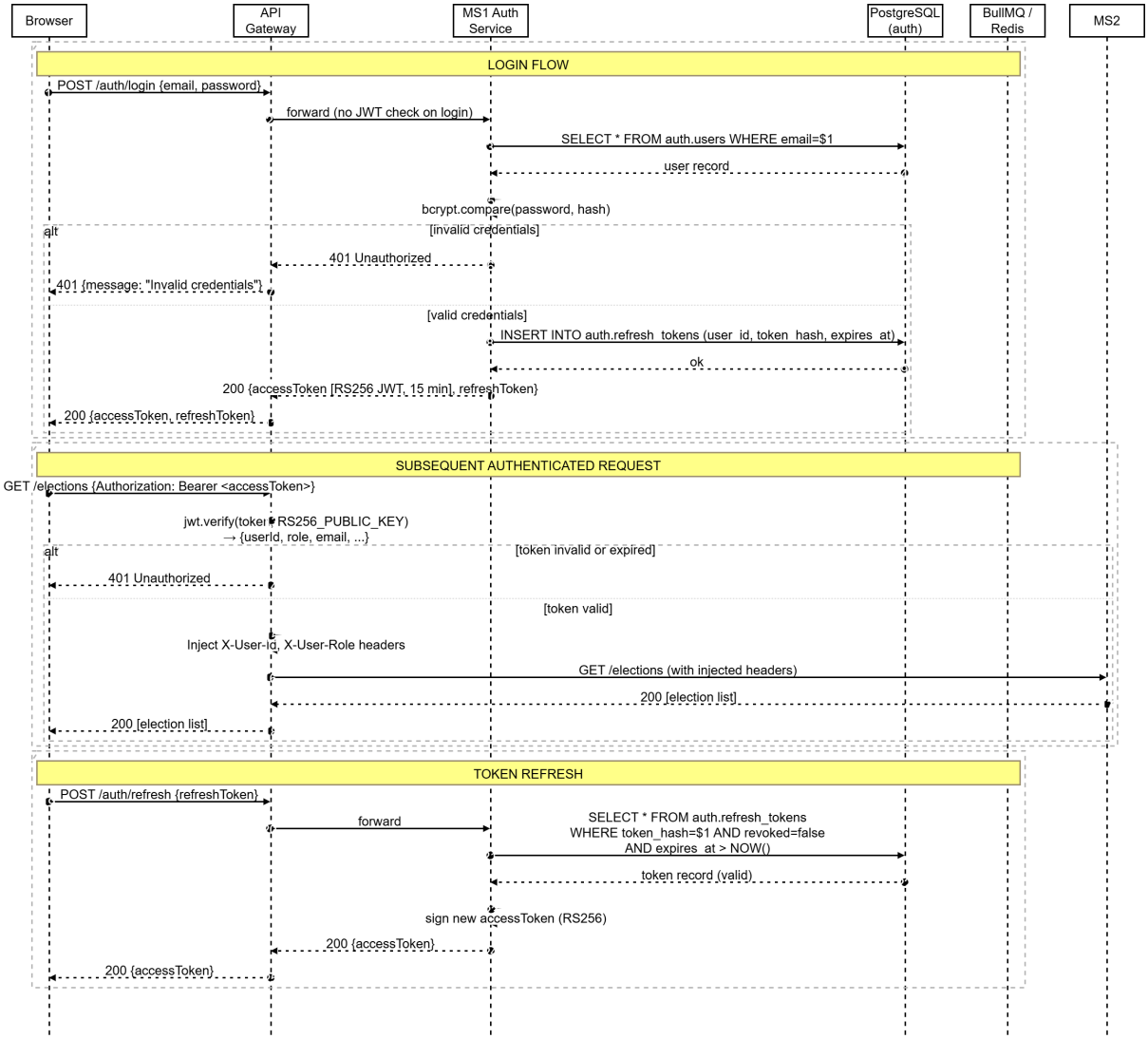

Figure 3: Auth Flow Sequence Diagram

3.2 MS2 — Election Service (Critical Service)

Owner: Syed Naimul Islam **Port:** 3002 **Database schema:** `election`

3.2.1 Responsibilities

MS2 is the most complex service. It manages the full election lifecycle (creation, candidate registration, voting, result publication), enforces concurrency-safe vote atomicity using `SELECT FOR UPDATE` transactions, implements result concealment until elections close, and schedules election open/close events using BullMQ delayed jobs.

3.2.2 Module Structure

Figure 4 shows the module diagram. `VoteService` is isolated from `ElectionService` because its concurrency requirements demand a different transaction strategy.

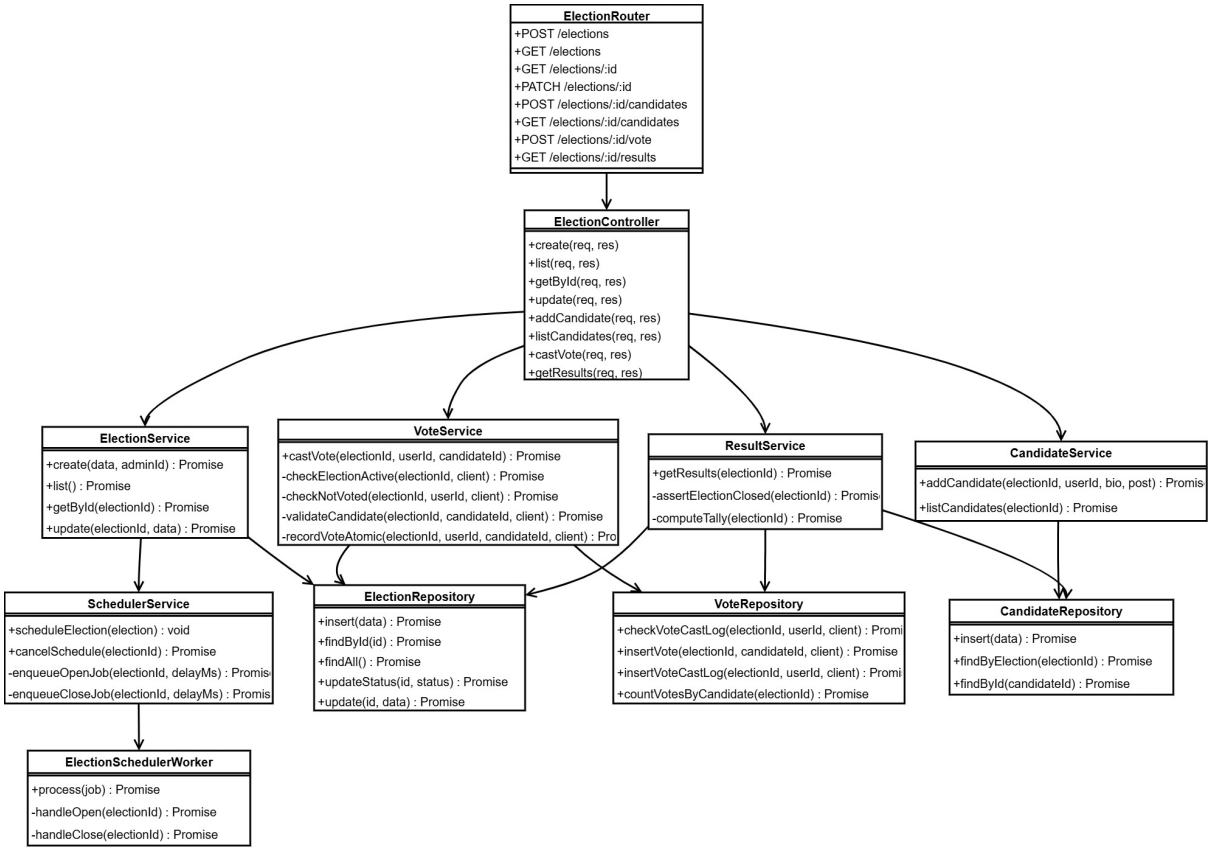

Figure 4: MS2 Module Diagram

3.2.3 API Endpoints

Method	Path	Auth	Role	Description
POST	`/elections`	Yes	Admin	Create new election
GET	`/elections`	No	—	List all elections
GET	`/elections/:id`	No	—	Get election details
PATCH	`/elections/:id`	Yes	Admin	Update election metadata
POST	`/elections/:id/candidates`	Yes	Admin	Add a candidate
GET	`/elections/:id/candidates`	Yes	Student+	List candidates with profiles
POST	`/elections/:id/vote`	Yes	Student	Cast a vote
GET	`/elections/:id/results`	Yes	Any	Get results (after close only)

3.2.4 Database Schema — `election`

Table: `election.elections`

Column	Type	Notes
election_id	SERIAL PK	—
title	VARCHAR(200)	Descriptive title
phase	SMALLINT	1 or 2
status	VARCHAR(20)	scheduled, active, closed
rules	TEXT	Free-text voting instructions
max_votes_per_user	SMALLINT	How many candidates a voter may select
start_time	TIMESTAMPTZ	Auto-open trigger
end_time	TIMESTAMPTZ	Auto-close trigger
created_by	INTEGER	Cross-schema ref: auth.users.user_id
created_at	TIMESTAMPTZ	—

Table: election.candidates

Column	Type	Notes
candidate_id	SERIAL PK	—
election_id	INTEGER FK	References election.elections
user_id	INTEGER	Cross-schema ref: auth.users.user_id
bio	TEXT	Short biography
post	VARCHAR(100)	Post being contested
created_at	TIMESTAMPTZ	—

Table: election.votes (stores *what* was voted — no voter identity)

Column	Type	Notes
vote_id	SERIAL PK	—
election_id	INTEGER FK	References election.elections
candidate_id	INTEGER FK	References election.candidates
cast_at	TIMESTAMPTZ	Timestamp of vote insertion

Table: election.vote_cast_log (stores *who* voted — no voting choice)

Column	Type	Notes
log_id	SERIAL PK	—
election_id	INTEGER FK	References election.elections
voter_user_id	INTEGER	Cross-schema ref: auth.users.user_id
voted_at	TIMESTAMPTZ	—

Vote anonymity design note: the votes table carries no voter_user_id; the vote_cast_log table carries no candidate_id. Even a direct database inspection by an administrator cannot link a voting choice to a specific voter, satisfying FR9 and NFR9.

3.2.5 Critical Algorithm — Vote Atomicity

This is the most important algorithm in the system. The goal is to ensure exactly-once vote recording under concurrent requests from the same or different users.

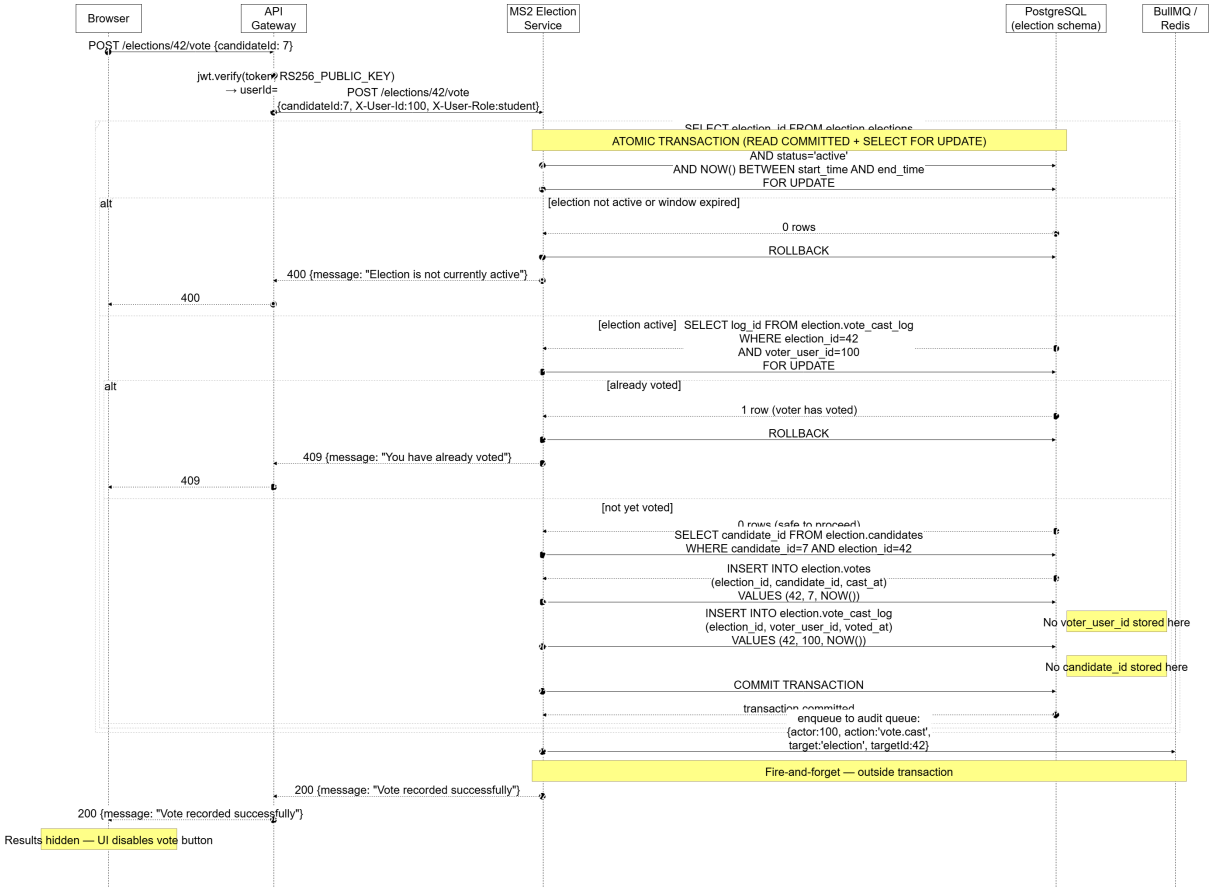

Figure 5: Vote Critical Path Sequence Diagram

The pseudocode for `VoteService.castVote(electionId, userId, candidateId)` is as follows:

Listing 1: Vote atomicity pseudocode (Node.js + PostgreSQL)

```
const client = await pool.connect();
try {
  await client.query('BEGIN');

  // Step 1: Lock the election row and verify it is active within its time window
  const electionRes = await client.query(
    `SELECT election_id FROM election.elections
     WHERE election_id = $1
       AND status = 'active'
       AND NOW() BETWEEN start_time AND end_time
     FOR UPDATE`,
    [electionId]
  );
```

```
  if (electionRes.rowCount === 0) {
    await client.query('ROLLBACK');
    return { status: 400, message: 'Election is not currently
 active' };
  }

  // Step 2: Check for a prior vote     lock the log row if it
 exists
  const castLogRes = await client.query(
    `SELECT log_id FROM election.vote_cast_log
     WHERE election_id = $1 AND voter_user_id = $2
     FOR UPDATE`,
    [electionId, userId]
  );
  if (castLogRes.rowCount > 0) {
    await client.query('ROLLBACK');
    return { status: 409, message: 'You have already voted in
 this election' };
  }

  // Step 3: Validate the candidate belongs to this election
  const candidateRes = await client.query(
    `SELECT candidate_id FROM election.candidates
     WHERE candidate_id = $1 AND election_id = $2`,
    [candidateId, electionId]
  );
  if (candidateRes.rowCount === 0) {
    await client.query('ROLLBACK');
    return { status: 404, message: 'Invalid candidate for this
 election' };
  }

  // Step 4: Insert the anonymous vote (no voter_user_id)
  await client.query(
    `INSERT INTO election.votes (election_id, candidate_id,
 cast_at)
     VALUES ($1, $2, NOW())`,
    [electionId, candidateId]
  );

  // Step 5: Record who voted (no candidate_id)
  await client.query(
    `INSERT INTO election.vote_cast_log (election_id,
 voter_user_id, voted_at)
     VALUES ($1, $2, NOW())`,
    [electionId, userId]
  );

  await client.query('COMMIT');

  // Step 6: Async audit log (outside transaction     fire-and-
```

```
   forget)
  auditQueue.add('audit.action', {
    actor: userId, action: 'vote.cast',
    target: 'election', targetId: electionId
  });

  return { status: 200, message: 'Vote recorded successfully' };
} catch (err) {
  await client.query('ROLLBACK');
  throw err;
} finally {
  client.release();
}
```

Concurrency guarantee: the `SELECT ... FOR UPDATE` on `vote_cast_log` in Step 2 places a row-level lock. Two simultaneous requests from the same user will serialize at the database level. The second transaction will block on the lock, then find the row inserted by the first and correctly return 409. No duplicate vote can be produced.

3.2.6 Algorithm — Election Auto-Scheduling

Elections open and close automatically, requiring no administrator intervention after creation. This is implemented using BullMQ's built-in delayed job support, which persists scheduled jobs in Redis and survives service restarts.

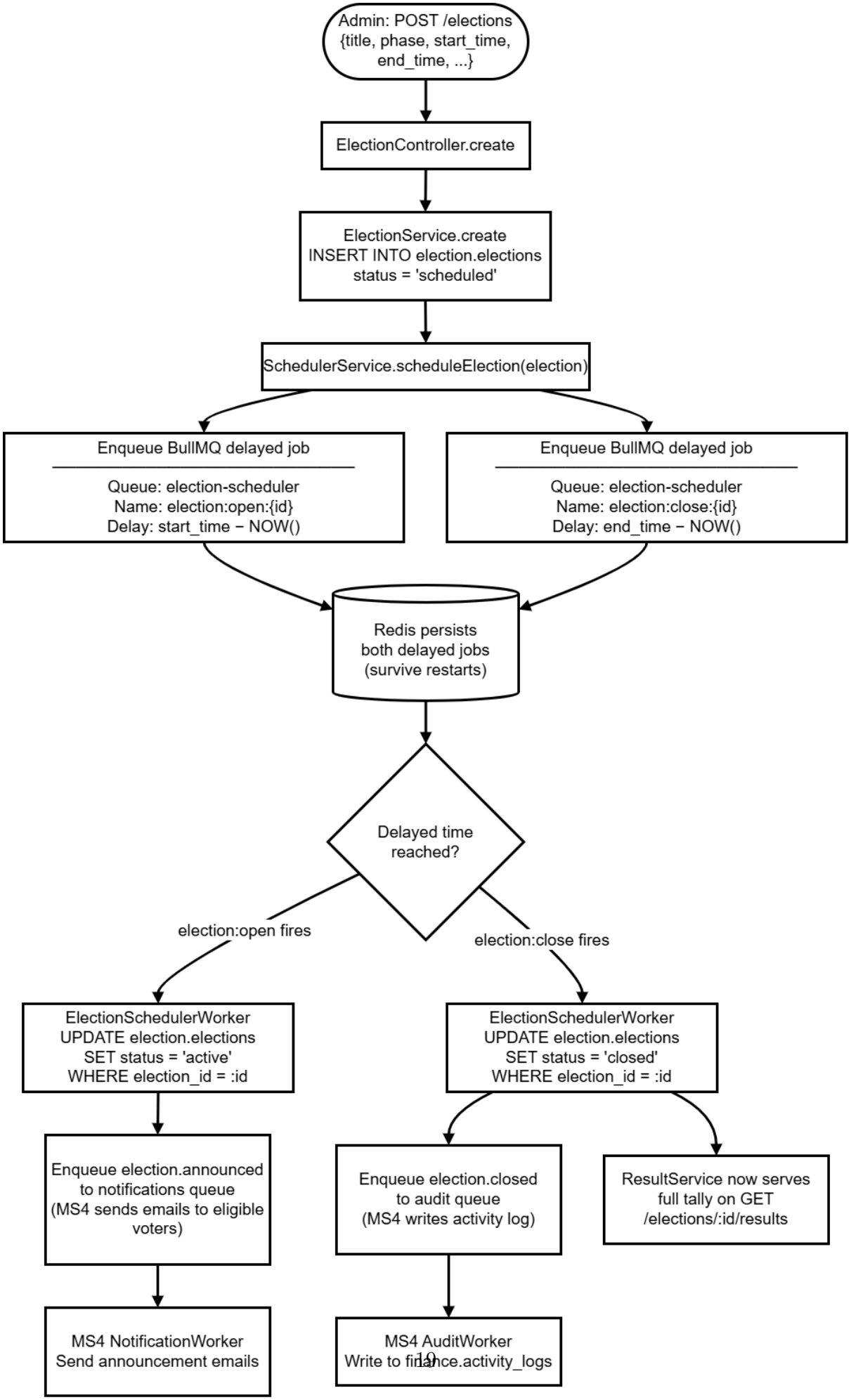

Figure 6: Election Scheduler Flowchart

On election creation:

1. `SchedulerService.scheduleElection(election)` is called after the DB insert.
2. Two BullMQ delayed jobs are enqueued on the `election-scheduler` queue:
   - `election:open:{id}` with delay = `start_time` − `NOW()`.
   - `election:close:{id}` with delay = `end_time` − `NOW()`.
3. The `ElectionSchedulerWorker` (running in MS2's process) handles both job types:
   - `election:open` fires a SQL update: `UPDATE election.elections SET status='active' WHERE election_id=$1`.
   - `election:close` fires: `UPDATE election.elections SET status='closed'`, then emits an `election.announced` event to the `notifications` queue (MS4 sends reminder emails).

Result concealment (FR12): `ResultService.getResults(electionId)` queries the election status before computing any tally. If `status` ≠ `'closed'`, it returns `403 Forbidden` with the message "Results will be available after the election closes." No tally data is returned, not even partial counts, eliminating any bandwagon effect.

3.3 MS3 — Event, Notice & Media Service

Owner: Md. Al Habib **Port:** 3003 **Database schema:** `content`

3.3.1 Responsibilities

MS3 manages the club's content layer: event creation and registration, volunteer coordination, the public notice board with priority and expiry, and media file uploads stored on the local filesystem.

3.3.2 Module Structure

Figure 7 shows the module diagram.

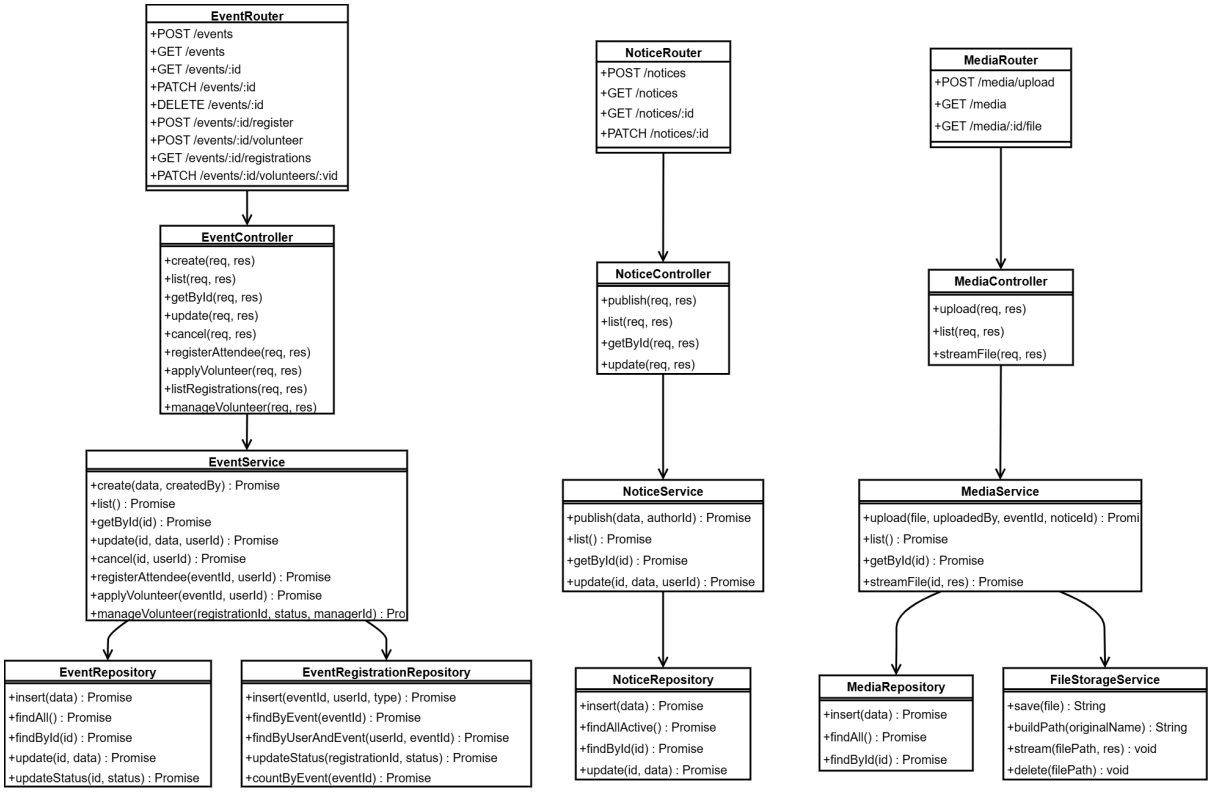

Figure 7: MS3 Module Diagram

3.3.3 API Endpoints

Events:

Method	Path	Auth	Role	Description
POST	`/events`	Yes	EC / Admin	Create event
GET	`/events`	No	—	List public events
GET	`/events/:id`	No	—	Get event detail
PATCH	`/events/:id`	Yes	EC / Admin	Update event
DELETE	`/events/:id`	Yes	EC / Admin	Cancel event
POST	`/events/:id/register`	Yes	Student	Register as attendee
POST	`/events/:id/volunteer`	Yes	Student	Apply to volunteer
GET	`/events/:id/registrations`	Yes	EC / Admin	List registrations
PATCH	`/events/:id/volunteers/:vid`	Yes	EC / Admin	Approve/reject volunteer

Notices:

Method	Path	Auth	Role	Description
POST	`/notices`	Yes	EC / Admin	Publish notice
GET	`/notices`	No	—	List active non-expired notices
GET	`/notices/:id`	No	—	Get single notice
PATCH	`/notices/:id`	Yes	EC / Admin	Update notice

Media:

Method	Path	Auth	Role	Description
POST	`/media/upload`	Yes	EC / Admin	Upload file
GET	`/media`	No	—	List all published media
GET	`/media/:id/file`	No	—	Stream / download a media file

3.3.4 Database Schema — `content`

Table: `content.events`

Column	Type	Notes
`event_id`	`SERIAL PK`	—
`title`	`VARCHAR(200)`	—
`description`	`TEXT`	—
`event_date`	`TIMESTAMPTZ`	—
`location`	`VARCHAR(300)`	—
`volunteers_needed`	`INTEGER`	Count of volunteer slots
`status`	`VARCHAR(20)`	`open`, `closed`, `cancelled`
`created_by`	`INTEGER`	Cross-schema ref: `auth.users.user_id`
`created_at`	`TIMESTAMPTZ`	—

Table: `content.event_registrations`

Column	Type	Notes
`registration_id`	`SERIAL PK`	—
`event_id`	`INTEGER FK`	References `content.events`

user_id	INTEGER	Cross-schema ref: auth.users.user_id
type	VARCHAR(20)	attendee or volunteer
status	VARCHAR(20)	pending, approved, rejected
registered_at	TIMESTAMPTZ	—

Table: content.notices

Column	Type	Notes
notice_id	SERIAL PK	—
title	VARCHAR(200)	—
content	TEXT	Body of the notice
priority	VARCHAR(10)	low, normal, urgent
expiry_date	DATE	Nullable; expired notices hidden from public board
created_by	INTEGER	Cross-schema ref: auth.users.user_id
published_at	TIMESTAMPTZ	—

Table: content.media

Column	Type	Notes
media_id	SERIAL PK	—
file_path	TEXT	Relative path on disk, e.g., /uploads/2026/04/{uuid}.jpg
file_type	VARCHAR(50)	MIME type
event_id	INTEGER	Nullable FK to content.events
notice_id	INTEGER	Nullable FK to content.notices
uploaded_by	INTEGER	Cross-schema ref: auth.users.user_id
uploaded_at	TIMESTAMPTZ	—

3.3.5 File Storage Design

Media files are persisted on the local filesystem via the **multer** middleware.

- **Storage path**: /var/uploads/{year}/{month}/{uuid}.{ext} where uuid is generated by crypto.randomUUID().
- **Max file size**: 10 MB (configurable via UPLOAD_MAX_SIZE_MB environment variable).
- **Allowed MIME types**: image/jpeg, image/png, image/gif, video/mp4, application/pdf.
- **Serving**: GET /media/:id/file streams the file using Node.js's fs.createReadStream() with appropriate Content-Type and Content-Disposition headers.

- **Notice expiry**: a lightweight BullMQ repeatable job (runs every hour) checks `content.notices` and hides expired notices. Alternatively, the `GET /notices` query applies a `WHERE expiry_date IS NULL OR expiry_date > CURRENT_DATE` filter — no background job required.

3.4 MS4 — Finance, Notification & Logging Service

Owner: Mahmud Hasan Walid **Port:** 3004 **Database schema:** `finance`

3.4.1 Responsibilities

MS4 owns two roles: it serves synchronous REST endpoints for budget management, and it runs long-lived BullMQ worker processes that consume events from other services to send emails and write audit logs.

3.4.2 Module Structure

Figure 8 shows the module diagram.

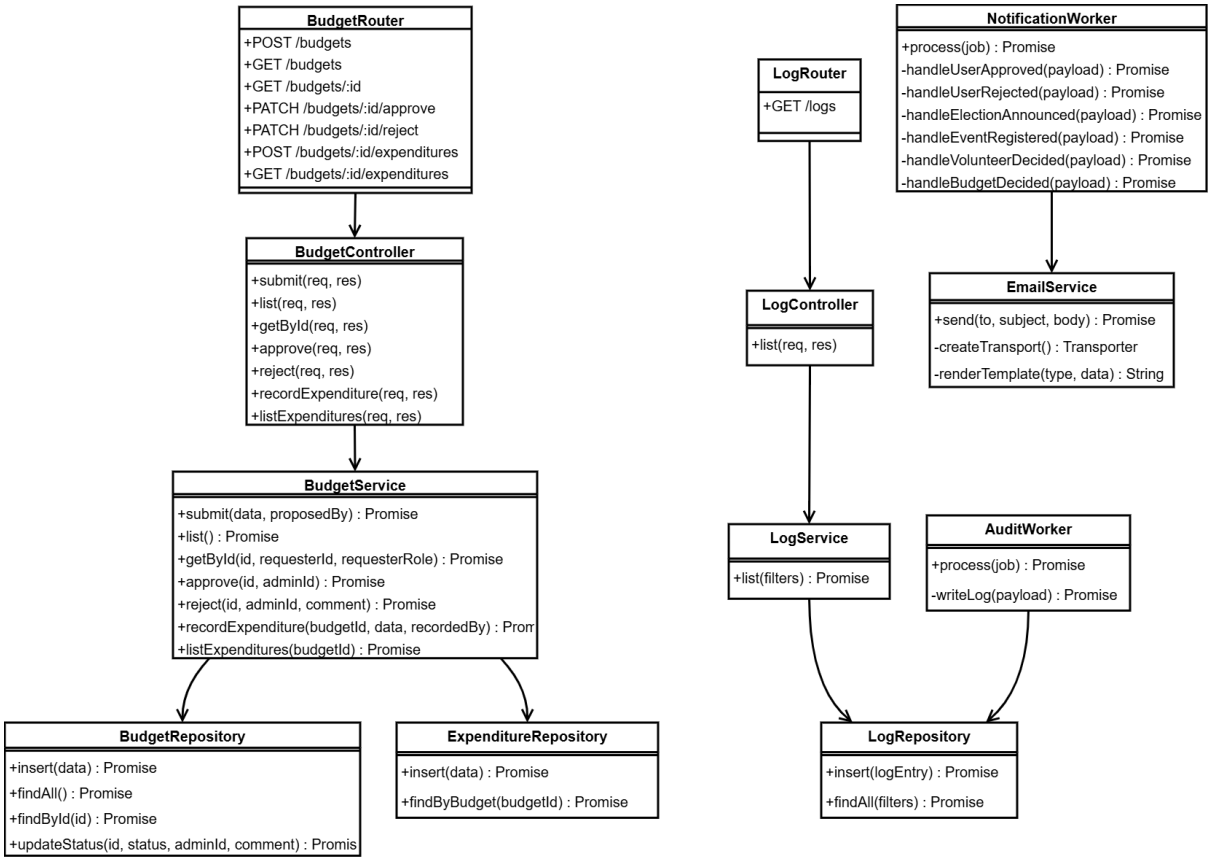

Figure 8: MS4 Module Diagram

3.4.3 API Endpoints

Method	Path	Auth	Role	Description
POST	`/budgets`	Yes	EC	Submit budget proposal
GET	`/budgets`	Yes	Admin	List all budget proposals
GET	`/budgets/:id`	Yes	EC (own) / Admin	Get budget detail
PATCH	`/budgets/:id/approve`	Yes	Admin	Approve budget
PATCH	`/budgets/:id/reject`	Yes	Admin	Reject with comment
POST	`/budgets/:id/expenditures`	Yes	EC / Admin	Record expenditure
GET	`/budgets/:id/expenditures`	Yes	Admin	List expenditures
GET	`/logs`	Yes	Admin	Get activity log (with filters)

3.4.4 Database Schema — `finance`

Table: `finance.budgets`

Column	Type	Notes
`budget_id`	`SERIAL PK`	—
`event_id`	`INTEGER`	Cross-schema ref: `content.events.event_id`
`proposed_by`	`INTEGER`	Cross-schema ref: `auth.users.user_id`
`status`	`VARCHAR(20)`	`pending_review`, `approved`, `rejected`
`total_amount`	`NUMERIC(12,2)`	Sum of line items
`line_items`	`JSONB`	Array of {`category`, `estimated_cost`} objects
`admin_comment`	`TEXT`	Rejection / approval note
`reviewed_by`	`INTEGER`	Cross-schema ref: `auth.users.user_id`; nullable
`submitted_at`	`TIMESTAMPTZ`	—
`reviewed_at`	`TIMESTAMPTZ`	Nullable; set on approve/reject

Table: `finance.expenditures`

Column	Type	Notes
`expenditure_id`	`SERIAL PK`	—
`budget_id`	`INTEGER FK`	References `finance.budgets`

category	VARCHAR(100)	e.g., "Venue", "Catering"
amount	NUMERIC(12,2)	Actual amount spent
description	TEXT	—
recorded_at	TIMESTAMPTZ	—
recorded_by	INTEGER	Cross-schema ref: auth.users.user_id

Table: finance.activity_logs

Column	Type	Notes
log_id	SERIAL PK	—
actor_user_id	INTEGER	Cross-schema ref: auth.users.user_id
action_type	VARCHAR(80)	e.g., user.approved, election.created
target_entity	VARCHAR(60)	e.g., user, election, budget
target_entity_id	INTEGER	ID of the affected record
details	JSONB	Additional metadata
logged_at	TIMESTAMPTZ	—

3.4.5 BullMQ Worker Design

MS4 starts two worker processes alongside its HTTP server:

- NotificationWorker: consumes the notifications queue. Each job carries a type field that dispatches to the appropriate email template:

Job type	Action
user.approved	Send approval email to the student
user.rejected	Send rejection email with reason
election.announced	Send announcement to all eligible voters
event.registered	Send confirmation email to registrant
volunteer.decided	Notify volunteer of approval/rejection
budget.decided	Notify EC member of budget approval/rejection

  Retry policy: **3 attempts** with exponential back-off (1 s, 5 s, 30 s).

- AuditWorker: consumes the audit queue. Each job carries { actor, action, target, targetId, details } and is written directly to finance.activity_logs. This is append-only; no update or delete is ever issued against this table.

Financial access control (FR23): BudgetService.getById checks the proposed_by field against the requesting user's ID (from the gateway-injected X-User-Id header). An EC member whose ID does not match proposed_by receives 403 Forbidden. Administrators bypass this check.

4 Database Design

4.1 Schema Overview

All four schemas reside in a single PostgreSQL 16 instance. Each microservice connects using a dedicated database user whose `SEARCH_PATH` is set to its own schema, preventing accidental cross-schema writes.

Schema	Owner (service)	Tables
`auth`	MS1	`users`, `refresh_tokens`, `password_reset_tokens`
`election`	MS2	`elections`, `candidates`, `votes`, `vote_cast_log`
`content`	MS3	`events`, `event_registrations`, `notices`, `media`
`finance`	MS4	`budgets`, `expenditures`, `activity_logs`

4.2 Cross-Schema References

Because all schemas share one PostgreSQL instance, services may issue read-only cross-schema queries where needed (e.g., `SELECT batch_year FROM auth.users WHERE user_id = $1` from within MS2). These references are enforced at the **application layer only** — no cross-schema `FOREIGN KEY` constraints are declared in the DDL, preserving logical service independence and allowing future migration to separate database instances with minimal code change.

4.3 Consolidated Entity-Relationship Diagram

Figure 9 shows all entities and their relationships across schemas. Schema-crossing references are shown as dashed arrows.

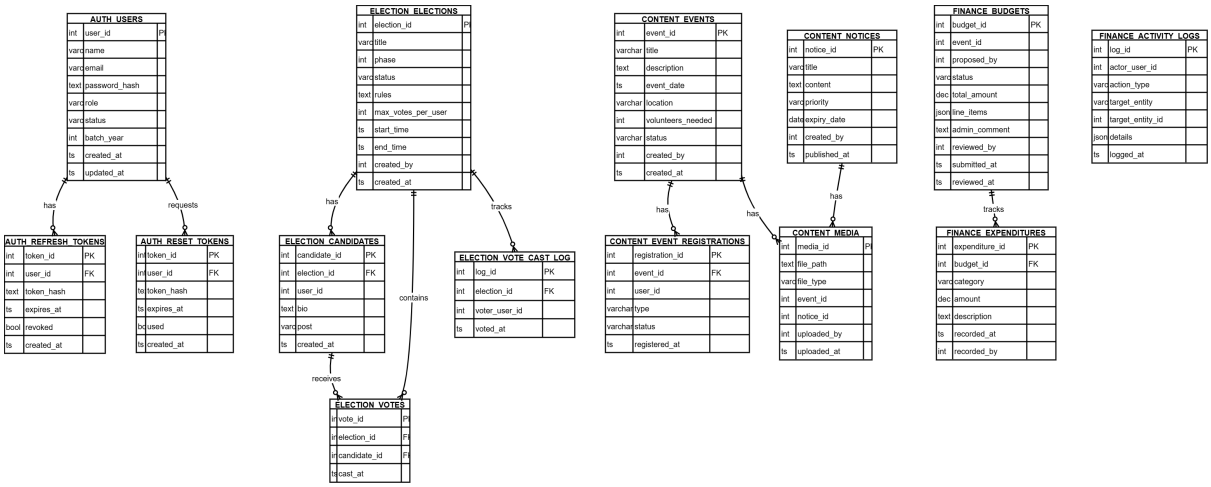

Figure 9: Consolidated ER Diagram (all schemas)

5 Asynchronous Communication

5.1 BullMQ Architecture

All asynchronous communication flows through two BullMQ queues hosted on Redis:

- `notifications`: carries email-sending jobs. Producers: MS1, MS2, MS3, MS4 (budget REST). Consumer: MS4 `NotificationWorker`.
- `audit`: carries audit log write jobs. Producers: MS1, MS2, MS3, MS4. Consumer: MS4 `AuditWorker`.

A third queue, `election-scheduler`, is internal to MS2 and is used exclusively for delayed open/close jobs (Section **??**). No other service reads from or writes to this queue.

5.2 Event Catalogue

Event Type	Queue	Producer	Payload (key fields)
`user.approved`	notifications	MS1	`userId, email, name`
`user.rejected`	notifications	MS1	`userId, email, name, reason`
`election.announced`	notifications	MS2 (scheduler)	`electionId, title, startTime, eligibleRoles`
`event.registered`	notifications	MS3	`userId, email, eventId, eventTitle`
`volunteer.decided`	notifications	MS3	`userId, email, eventId, status`
`budget.decided`	notifications	MS4	`userId, email, budgetId, status, comment`
`audit.action`	audit	MS1–MS4	`actor, action, target, targetId, details`

5.3 BullMQ Event Flow Diagram

Figure 10 shows the end-to-end flow from producer services through Redis to the MS4 workers and their outputs.

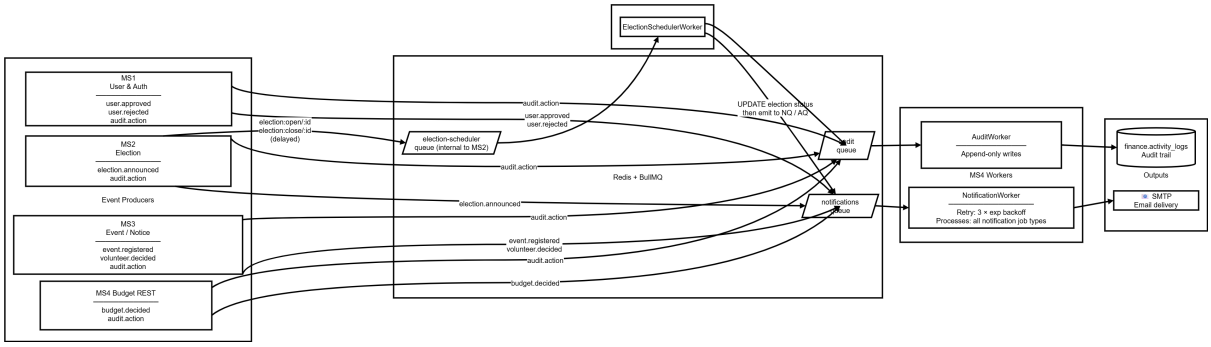

Figure 10: BullMQ Event Flow Diagram

6 Security Design

6.1 JWT Authentication Flow

1. On successful login, MS1 signs an access token with its **RS256 private key** (stored only in MS1's environment). The token carries `sub`, `role`, `email`, `name`, and `batch_year`.
2. The **RS256 public key** is distributed to the API Gateway as a `JWT_PUBLIC_KEY` environment variable at deploy time.
3. The Gateway verifies every incoming Bearer token using the public key. No network call to MS1 is made per request. This is the key architectural benefit of RS256 over HS256 in a microservices context.
4. On token expiry (15 min), the client calls `POST /auth/refresh` with its refresh token. MS1 looks up the token hash in `auth.refresh_tokens`, validates expiry and revocation status, then issues a new access token.

6.2 RBAC Implementation

Role-based access control is enforced at two levels:

- **API Gateway level**: the Gateway rejects requests to admin-only route prefixes (`/users/*`, `/logs/*`) if the JWT role claim is not `admin`, before the request ever reaches a microservice.
- **Service level**: each service also checks the `X-User-Role` header on sensitive operations (e.g., `BudgetService` verifying that only `ec_member` can submit proposals, only `admin` can approve). This defence-in-depth ensures correctness even if the Gateway is bypassed on the internal network.

Action	Visitor	Student	EC Member	Admin

View events, notices, media	✓	✓	✓	✓
Login / register	✓	✓	✓	✓
Vote in election	—	✓	✓	✓
Register for / volunteer at events	—	✓	✓	✓
Create events, publish notices	—	—	✓	✓
Submit budget proposals	—	—	✓	✓
Approve / reject registrations	—	—	—	✓
Approve / reject budgets	—	—	—	✓
Manage elections	—	—	—	✓
View activity logs	—	—	—	✓
View all financial records	—	—	—	✓
Promote / revoke users	—	—	—	✓

6.3 Vote Anonymity Design

The separation of the `votes` table (what was voted) from the `vote_cast_log` table (who voted) ensures that no single SQL query can reconstruct the mapping from voter to candidate. Even if an administrator executes `SELECT * FROM election.votes` they will see candidate IDs and timestamps but no voter IDs. Even if they join `vote_cast_log` they will see who voted and when but not which candidate. The two tables are correlated only by election and timestamp — not by a shared key — making individual vote tracing computationally infeasible in a large election.

6.4 Additional Security Measures

- **Password hashing**: bcrypt with cost factor 12 (NFR6). Plaintext passwords never appear in logs.
- **Input validation**: `express-validator` is used in all services. All database queries use parameterised statements (`pg` library), preventing SQL injection (NFR7).
- **HTTPS**: in production, Nginx terminates TLS in front of the Gateway. All internal Docker network communication remains HTTP (trusted network boundary).
- **Rate limiting**: the vote endpoint is rate-limited at the Gateway to 30 requests per minute per IP, mitigating automated attack attempts.

7 Deployment Architecture

7.1 Docker Compose Layout

The entire system is orchestrated with a single `docker-compose.yml` file defining nine services: `postgres`, `redis`, `gateway`, `ms1-auth`, `ms2-election`, `ms3-content`, `ms4-finance`, and a shared Docker volume for media uploads.

Listing 2: Abbreviated Docker Compose service definitions

```yaml
services:
  postgres:
    image: postgres:16
    environment:
      POSTGRES_DB: cseduclub
      POSTGRES_USER: pgadmin
      POSTGRES_PASSWORD: ${DB_PASSWORD}
    volumes:
      - pg_data:/var/lib/postgresql/data
      - ./db/init.sql:/docker-entrypoint-initdb.d/init.sql

  redis:
    image: redis:7-alpine

  gateway:
    build: ./gateway
    ports: ["3000:3000"]
    environment:
      JWT_PUBLIC_KEY: ${JWT_PUBLIC_KEY}
      MS1_URL: http://ms1-auth:3001
      MS2_URL: http://ms2-election:3002
      MS3_URL: http://ms3-content:3003
      MS4_URL: http://ms4-finance:3004
    depends_on: [ms1-auth, ms2-election, ms3-content, ms4-finance]

  ms1-auth:
    build: ./ms1-auth
    environment:
      DB_URL: postgres://ms1user:${MS1_DB_PASS}@postgres/cseduclub
      JWT_PRIVATE_KEY: ${JWT_PRIVATE_KEY}
      JWT_PUBLIC_KEY:  ${JWT_PUBLIC_KEY}
      REDIS_URL: redis://redis:6379

  ms2-election:
    build: ./ms2-election
    environment:
      DB_URL: postgres://ms2user:${MS2_DB_PASS}@postgres/cseduclub
      REDIS_URL: redis://redis:6379
```

```yaml

  ms3-content:
    build: ./ms3-content
    environment:
      DB_URL: postgres://ms3user:${MS3_DB_PASS}@postgres/cseduclub
      REDIS_URL: redis://redis:6379
      UPLOAD_DIR: /var/uploads
    volumes: [media_uploads:/var/uploads]

  ms4-finance:
    build: ./ms4-finance
    environment:
      DB_URL: postgres://ms4user:${MS4_DB_PASS}@postgres/cseduclub
      REDIS_URL: redis://redis:6379
      SMTP_HOST: ${SMTP_HOST}
      SMTP_USER: ${SMTP_USER}
      SMTP_PASS: ${SMTP_PASS}

volumes:
  pg_data:
  media_uploads:
```

7.2 Service Port Summary

Service	Port	Notes
API Gateway	3000	Only port exposed to host / internet
MS1 — User & Auth	3001	Internal Docker network only
MS2 — Election	3002	Internal Docker network only; scale with `--scale ms2-election=3`
MS3 — Event / Notice / Media	3003	Internal Docker network only
MS4 — Finance / Notify / Log	3004	Internal Docker network only
PostgreSQL	5432	Internal only; one instance, four schemas
Redis	6379	Internal only; BullMQ queues + optional token cache

7.3 Database Initialisation

A single `db/init.sql` script (run by the `postgres` container on first start) creates the four schemas, dedicated per-service database users, and grants:

Listing 3: Schema and user setup (init.sql excerpt)

```sql
CREATE SCHEMA IF NOT EXISTS auth;
CREATE SCHEMA IF NOT EXISTS election;
```

```sql
CREATE SCHEMA IF NOT EXISTS content;
CREATE SCHEMA IF NOT EXISTS finance;

CREATE USER ms1user WITH PASSWORD '...';
GRANT USAGE  ON SCHEMA auth     TO ms1user;
GRANT ALL    ON ALL TABLES IN SCHEMA auth     TO ms1user;

-- ms2user gets USAGE on auth for cross-schema reads
CREATE USER ms2user WITH PASSWORD '...';
GRANT USAGE  ON SCHEMA election TO ms2user;
GRANT ALL    ON ALL TABLES IN SCHEMA election TO ms2user;
GRANT USAGE  ON SCHEMA auth     TO ms2user;
GRANT SELECT ON auth.users      TO ms2user;

-- similar grants for ms3user, ms4user ...
```

7.4 Failure Isolation

Because each microservice is a separate Docker container, a crash in one service does not affect others. During an active election, if MS3 (content) or MS4 (finance) crashes:

- The API Gateway continues routing `/elections/*` requests to the still-running MS2 container.
- Voting proceeds uninterrupted.
- BullMQ jobs persist in Redis; MS4 workers replay them once the container restarts, ensuring no emails or audit logs are lost.

8 Document Revision History

Version	Date	Description	Author
1.0	April 2026	Initial SDD release. Covers microservice decomposition, API contracts, database schemas, voting concurrency algorithm, election scheduler design, BullMQ event catalogue, RBAC table, vote anonymity design, and Docker Compose deployment layout.	Project Team